**SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**

**Department of Computer Science and Engineering**

**CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise**

| | | | |
|---|---|---|---|
| **Course Code:** | CSA10 / CSA1063 | **Course Title:** | Software Engineering |
| **CO Assessed:** | CO4 | **Assessment:** | Assessment Tool 2 – CI/CD Pipeline Design Exercise |
| **Weightage:** | 20% | **Total Marks:** | 25 |
| **CO4 Statement:** | Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics | | |
| **Student Name:** | | **Reg. No.:** | |
| **Date:** | | **Duration:** | 60 minutes |

**Code of Conduct**

*I Y.HEMA SAI LAHARI(192311446)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Y.HEMA SAI LAHARI

**Annexure A – CI/CD Pipeline Design Exercise**

**Instructions to Students:**

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

**Deliverable:** Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

**CI/CD PIPELINE DESIGN DOCUMENT**

**TEMPLE DONATION AND EVENT MANAGEMENT SYSTEM**

---

**1. Introduction**

The **Temple Donation and Event Management System** is a web-based application developed to manage temple donations, devotees, events, registrations, payment records, and administrative activities.

A **CI/CD (Continuous Integration and Continuous Deployment) pipeline** is designed to automate the processes of source-code integration, building, testing, code-quality verification, security checking, packaging, and deployment.

The proposed pipeline helps ensure that every code change is automatically validated before being deployed to the application environment.

---

**2. Objectives**

The main objectives of the CI/CD pipeline are:

- Automate code integration and deployment.
- Detect coding errors at an early stage.
- Automatically execute software tests.
- Maintain code quality and security.
- Reduce manual deployment effort.
- Provide faster and reliable software releases.
- Maintain separate development, staging, and production environments.
- Provide rollback support when deployment failures occur.
- Ensure that only tested and approved code reaches production.

---

**3. Project Requirements**

The system should provide the following major functionalities:

1. User registration and login.
2. User profile management.
3. Temple information.
4. Donation category management.
5. Online donation/payment processing.
6. Donation receipt generation.
7. Donation history.
8. Event creation and management.
9. Event registration.
10. Admin dashboard.
11. User and donation management.
12. Report generation.
13. Secure authentication and authorization.

Because the application handles **user information, donation records, and payment-related operations**, the CI/CD pipeline should include strong testing, security checks, and controlled production deployment.

---

**4. CI/CD Requirements**

The pipeline should:

- Automatically trigger when code is pushed to the repository.
- Build the application automatically.
- Check source-code quality.
- Run unit tests.
- Run integration tests.
- Perform security checks.
- Package the application.
- Deploy successful builds to the development environment.
- Deploy approved builds to staging.
- Deploy verified builds to production.
- Monitor deployment status.
- Provide rollback if deployment fails.

---

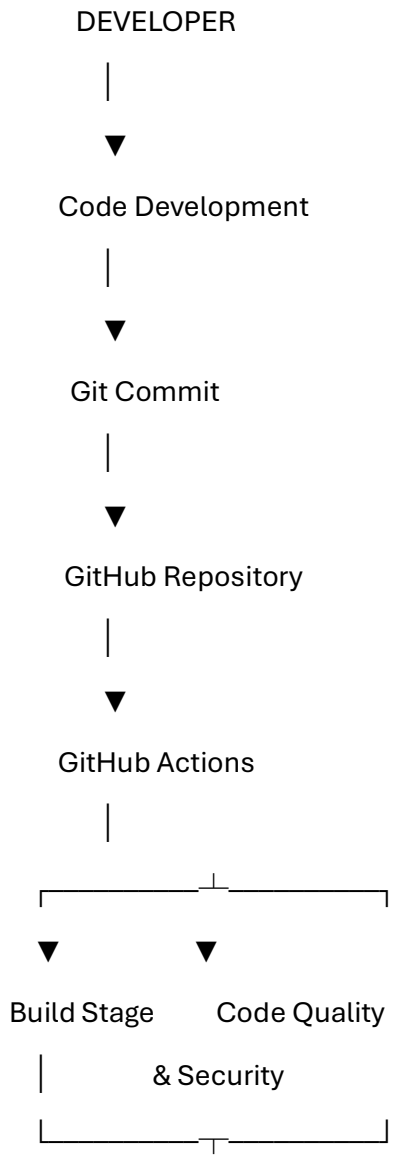
**5. Tools and Technologies**

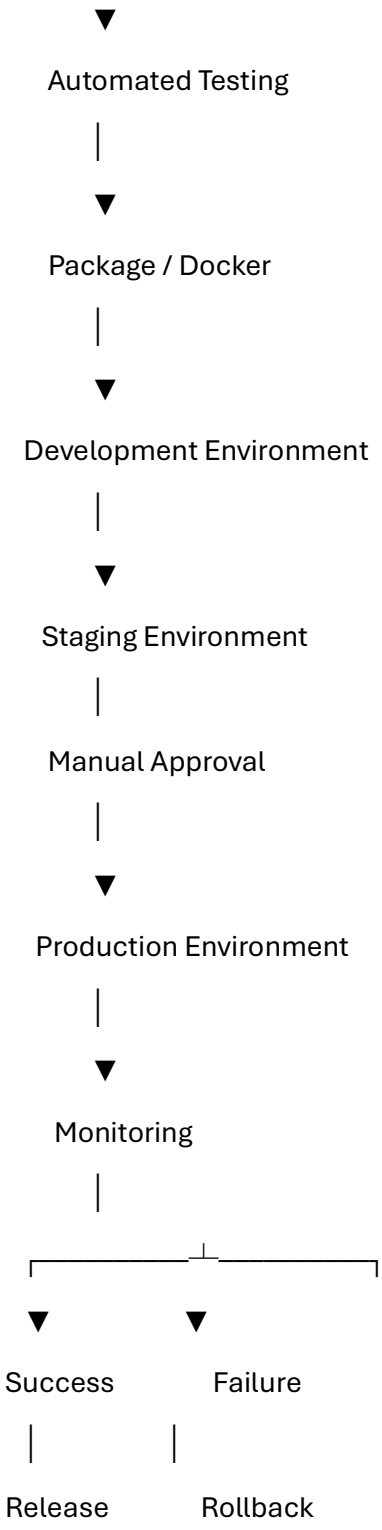
| Pipeline Stage | Tool/Technology | Purpose |
|---|---|---|
| Source Code Management | Git | Version control |
| Repository | GitHub | Store and manage source code |
| CI/CD | GitHub Actions | Automate pipeline |
| Frontend Build | npm / Vite | Build frontend application |
| Backend | Node.js | Run backend application |
| Testing | Jest / React Testing Library | Automated testing |
| API Testing | Postman/Newman | API validation |
| Code Quality | SonarQube | Static code analysis |
| Security | npm audit / OWASP tools | Dependency/security checks |

| Pipeline Stage | Tool/Technology | Purpose |
|---|---|---|
| Containerization | Docker | Package application |
| Deployment | Docker / Cloud platform | Application deployment |
| Database | MongoDB | Store application data |
| Notifications | GitHub Actions notifications | Build/deployment status |
| Monitoring | Application logs/monitoring tools | Detect deployment/runtime issues |

---

**6. Proposed CI/CD Pipeline Architecture**






---

**7. Pipeline Workflow**

**Stage 1 – Source Code Management**

Developers work on the Temple Donation and Event Management System and maintain the source code using **Git**.

The project repository is hosted on **GitHub**.

**Activities:**

- Create feature branches.
- Develop new features.
- Commit changes.
- Push changes to GitHub.
- Create Pull Requests.
- Review code before merging.

**Example:**

feature/donation-module

feature/event-management

feature/user-authentication

---

**8. Stage 2 – Continuous Integration Trigger**

Whenever a developer pushes code or creates a Pull Request, **GitHub Actions** automatically starts the CI pipeline.

**Trigger conditions:**

- Push to development branch.
- Pull Request creation.
- Pull Request update.
- Merge into main branch.

The pipeline ensures that new code does not break existing functionality.

---

**9. Stage 3 – Build**

The application dependencies are installed and the application is built.

**Activities:**

1. Checkout source code.

2. Install Node.js.
3. Install dependencies.
4. Build frontend.
5. Build backend.
6. Check for build errors.

Example commands:

npm install

npm run build

**Expected Result**

The application should compile successfully without build errors.

---

**10. Stage 4 – Automated Testing**

Automated testing is performed after successful compilation.

**Testing levels:**

**Unit Testing**

Tests individual functions and components.

Examples:

- Login validation.
- Donation amount validation.
- Event registration validation.
- User input validation.

**Integration Testing**

Tests interaction between application components.

Examples:

- Frontend and backend communication.
- Backend and database communication.
- Donation transaction and receipt generation.

**API Testing**

API endpoints are tested using tools such as Postman/Newman.

Examples:

POST /login

POST /donations

GET /donations

POST /events

POST /events/register

**Regression Testing**

Existing functionality is tested whenever new features are introduced.

---

**11. Stage 5 – Code Quality Analysis**

**SonarQube** can be integrated into the pipeline to identify code-quality problems.

It checks for:

- Bugs.
- Code smells.
- Duplicated code.
- Maintainability issues.
- Security vulnerabilities.
- Poor coding practices.

The pipeline should stop if critical quality conditions are not satisfied.

---

**12. Stage 6 – Security Testing**

Security checks are important because the system handles user accounts and donation information.

**Security checks include:**

- Dependency vulnerability scanning.
- Detection of vulnerable packages.
- Secret/credential checking.

- Authentication testing.
- Authorization testing.
- Input validation.
- Basic API security checks.

Example:

npm audit

Sensitive information such as:

Database passwords

API keys

Payment credentials

JWT secrets

should not be stored directly in source code.

They should be maintained using **GitHub Secrets** or an appropriate secret-management system.

---

**13. Stage 7 – Packaging**

After successful testing and quality checks, the application is packaged for deployment.

**Docker** can be used to create a consistent application container.

Example:

Source Code

↓

Docker Build

↓

Docker Image

↓

Container Registry

The Docker image contains the required application dependencies and configuration required to run the application.

---

**14. Stage 8 – Development Environment**

The first deployment environment is the **Development Environment**.

It is used by developers to verify the newly built version.

**Activities:**

- Deploy successful build.
- Test newly implemented features.
- Check application logs.
- Verify database connectivity.
- Perform basic functional testing.

Only builds that successfully pass CI checks should be deployed.

---

**15. Stage 9 – Staging Environment**

After successful development validation, the application is deployed to the **Staging Environment**.

The staging environment should closely resemble production.

**Testing includes:**

- Complete functional testing.
- Integration testing.
- Regression testing.
- API testing.
- Security validation.
- User acceptance testing.

The staging environment allows the team to identify problems before production deployment.

---

**16. Stage 10 – Production Deployment**

After successful staging validation, the application can be deployed to the **Production Environment**.

A manual approval step can be added before production deployment.

Staging Testing

↓

Approval

↓

Production Deployment

This prevents accidental deployment of unverified code.

---

**17. Deployment Strategy**

The proposed deployment strategy consists of three environments:

| Environment | Purpose | Deployment |
|---|---|---|
| Development | Developer testing | Automatic |
| Staging | Final testing and validation | Automatic/Controlled |
| Production | Live application | Manual approval |

This approach reduces the risk of deploying defective code directly to production.

---

**18. Rollback Strategy**

A rollback mechanism is required if production deployment fails.

**Rollback process:**

New Version Deployed

↓

Health Check

↓

Successful?

↙ ↘

Yes No

↓ ↓

Continue  Rollback

↓

Previous Stable Version

If the new version causes serious problems, the system can return to the previously working version.

Docker images or versioned releases can be used to maintain previous stable versions.

---

**19. Automated Testing Strategy**

The following testing strategy is integrated into the CI/CD pipeline:

| Testing Type | Purpose |
|---|---|
| Unit Testing | Verify individual functions |
| Integration Testing | Verify module interaction |
| API Testing | Verify backend APIs |
| Regression Testing | Ensure existing features continue working |
| Security Testing | Identify security vulnerabilities |
| Performance Testing | Check application response |
| UI Testing | Verify important user workflows |

**Important automated workflows:**

**User Registration**

Open Registration

→ Enter Details

→ Submit

→ Verify Account Creation

**Donation**

Login

→ Select Donation Category

→ Enter Amount

→ Payment

→ Verify Transaction

→ Verify Receipt

**Event Registration**

Login

→ View Event

→ Register

→ Verify Registration

---

**20. Quality Gates**

Quality gates are conditions that must be satisfied before moving to the next stage.

| Quality Gate | Condition |
|---|---|
| Build Gate | Build must succeed |
| Unit Test Gate | Tests must pass |
| Code Quality Gate | No critical quality issues |
| Security Gate | No critical vulnerabilities |
| Integration Test Gate | Integration tests must pass |
| Staging Gate | Application must function correctly |
| Deployment Gate | Health check must pass |

If a quality gate fails, the pipeline should stop and notify the development team.

---

**21. Notifications**

The CI/CD pipeline should notify developers about:

- Successful builds.
- Failed builds.
- Failed automated tests.
- Security issues.

- Deployment success.
- Deployment failure.
- Rollback events.

Notifications can be integrated with email or team communication platforms.

---

**22. Security Measures**

Security should be incorporated throughout the pipeline.

**Measures:**

1. Use HTTPS for application communication.
2. Store secrets securely.
3. Never commit passwords or API keys.
4. Scan dependencies for vulnerabilities.
5. Use role-based access control.
6. Protect the main branch.
7. Require Pull Request reviews.
8. Perform security testing before production.
9. Use secure database credentials.
10. Restrict production deployment permissions.

---

**23. Sample GitHub Actions Workflow**

A simplified workflow can be designed as follows:

name: Temple Donation CI/CD

on:

push:

branches: [main]

pull_request:

branches: [main]

```
jobs:

  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: Install Dependencies
        run: npm install

      - name: Run Tests
        run: npm test

      - name: Build Application
        run: npm run build

      - name: Security Audit
        run: npm audit --audit-level=high
```

This workflow automatically checks the application whenever code is pushed or a Pull Request is created.

---

**24. Complete CI/CD Flow**

The complete workflow for the Temple Donation and Event Management System is:

Developer

↓

Git Repository

↓

Pull Request

↓

Code Review

↓

GitHub Actions

↓

Dependency Installation

↓

Build

↓

Unit Testing

↓

Integration/API Testing

↓

Code Quality Analysis

↓

Security Scan

↓

Docker Packaging

↓

Development Deployment

↓

Staging Deployment

↓

Acceptance & Regression Testing

↓

Manual Approval

↓

Production Deployment

↓

Health Check

↓

Monitoring

↓

Success / Rollback

---

**25. Advantages of the Proposed CI/CD Pipeline**

- Faster software delivery.
- Reduced manual effort.
- Early detection of defects.
- Automated testing.
- Improved code quality.
- Improved security.
- Consistent deployment.
- Reduced production failures.
- Easy rollback.
- Better collaboration between developers and testers.
- Continuous monitoring of application health.

---

**26. Justification of Tool Selection**

**Git and GitHub**

Git provides version control, while GitHub provides repository hosting, Pull Requests, collaboration, and source-code management.

**GitHub Actions**

GitHub Actions is suitable because it can directly automate build, testing, security checks, and deployment workflows within the GitHub repository.

**Jest**

Jest provides automated JavaScript testing and can be integrated directly into the CI pipeline.

**SonarQube**

SonarQube helps identify bugs, code smells, duplicated code, and security-related code-quality issues.

**Docker**

Docker provides consistent packaging and helps ensure that the application behaves similarly across different environments.

**npm Audit**

It helps identify known vulnerabilities in project dependencies.

---

**27. Expected Outcome**

After implementing the proposed CI/CD pipeline, every code change will pass through automated validation before reaching production.

The pipeline ensures:

**Code → Build → Test → Quality → Security → Package → Deploy → Monitor → Rollback if Required**

This provides a reliable and controlled development and deployment process for the **Temple Donation and Event Management System**.

---

**28. Conclusion**

The proposed CI/CD pipeline automates the complete software delivery process for the Temple Donation and Event Management System. It integrates source-code

management, automated building, testing, code-quality analysis, security verification, packaging, deployment, monitoring, and rollback mechanisms.

Using **GitHub, GitHub Actions, automated testing tools, SonarQube, Docker, and security scanning**, the system can achieve faster, safer, and more reliable software releases. The use of separate **development, staging, and production environments** ensures that only verified and approved software is released to end users.

**Rubrics for Calculation**

| Evaluation Criteria | Excellent (4 Marks) | Good (3 Marks) | Satisfactory (2 Marks) | Needs Improvement (1 Mark) |
|---|---|---|---|---|
| **Requirement & Pipeline Analysis(15%)** | Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements | Identifies most relevant requirements | Identifies basic requirements with some gaps | Requirements are unclear or incomplete |
| **Pipeline Architecture & Workflow(25%)** | Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages | Pipeline covers most major stages with minor gaps | Basic pipeline is designed but several stages are missing | Pipeline is incomplete or poorly structured |

| | | | | |
|---|---|---|---|---|
| **Tools & Technology Selection(20%)** | Selects appropriate tools for each stage and provides clear justification | Appropriate tools selected with reasonable justification | Tools are identified but justification is limited | Tools are inappropriate or not clearly identified |
| **Automated Testing & Quality Integration(15%)** | Effectively integrates automated testing and code-quality/security checks into the pipeline | Includes major testing and quality checks | Includes basic testing with limited integration | Testing/quality checks are missing or inappropriate |
| **Deployment, Security & Rollback Strategy(15%)** | Clearly designs deployment environments, security practices, monitoring, and rollback strategy | Covers deployment and most security/rollback aspects | Basic deployment strategy with limited security/rollback details | Deployment strategy is incomplete or lacks security considerations |
| **Pipeline Diagram & Documentation(10%)** | Clear, accurate, professional diagram with excellent explanation and documentation | Well-presented diagram and documentation with minor issues | Adequate diagram/documentation but lacks clarity | Poorly presented, incomplete, or difficult to understand |

| Criteria | **Mark** |
|---|---|
| **Requirement & Pipeline Analysis(15%)** | /5 |
| **Pipeline Architecture & Workflow(25%)** | /4 |
| **Tools & Technology Selection(20%)** | /4 |
| **Automated Testing & Quality Integration(15%)** | /4 |
| **Deployment, Security & Rollback Strategy(15%)** | /4 |
| **Pipeline Diagram & Documentation(10%)** | /4 |
| **TOTAL** | /25 |